Week 05 — IoT Communication Protocols and Messaging

> Slides: week05-iot-edge-05.pdf

Bridge from the previous weeks

Week 3	Week 4	Week 5
Prepare the data	**Establish** connectivity	**Communicate** the data

Connectivity exists — now we must decide: **where should the data go?**, **which application/process should receive it?**, **how should data be transported?**, and **how should the application exchange messages?**

Learning objectives: explain IP communication, explain **TCP and UDP**, explain **request-response and publish-subscribe**, describe **HTTP, CoAP and MQTT**, trace IoT data through the protocol stack, and select suitable communication protocols.

Contents: 1. IP and transport · 2. Request-response communication · 3. MQTT and publish-subscribe

---

Part 1 — IP and Transport

The communication stack: one protocol cannot do everything

Layer	Examples	Main question
Connectivity / link	Wi-Fi, Ethernet, IEEE 802.15.4	**How does the device access the network?**
Network protocol	**IP**	**Where should the data go?**
Transport protocol	**TCP, UDP**	**Which application should receive it? How should it be delivered?**
Application protocol	**HTTP, CoAP, MQTT**	**How should applications exchange information?**

> **Package-delivery analogy:** the **network protocol** is the university and building (**IP** → **which host?**); the **transport protocol** is the room number (**port** → **which application/process?**); the **application protocol** is the rules governing content and interaction (**HTTP/CoAP/MQTT**).

In one line: IP finds the device, TCP/UDP delivers data to the correct application, and HTTP/CoAP/MQTT define how applications exchange meaningful messages.

IP address

> **IP address** = a **logical address** used to identify a **device/interface** on an IP network.

Main purposes: identify the **source**, identify the **destination**, and **allow data to be delivered across interconnected networks**.

> **Connectivity technology establishes access to a network; IP enables communication *across* networks.**

University example: office PC `10.20.35.47` → university local network (Ethernet) → university router / **default gateway** `10.20.35.1` → routers forward packets across interconnected networks → remote server `203.0.113.10`.

```
Example IP packet:  Source IP: 10.20.35.47   Destination IP: 203.0.113.10
```

If the destination is **outside the local network**, traffic is sent to the **default gateway**.

IPv4

- Example: **192.168.1.25** — **32 bits**, written as **4 decimal numbers** of 8 bits each.
- `192 = 11000000`, `168 = 10101000`, `1 = 00000001`, `25 = 00011001` → `11000000.10101000.00000001.00011001`
- It identifies a **network interface locally**, and the **address may change** depending on network configuration.

Local vs remote communication

Case A — Local	**Case B — Remote**
Sensor `192.168.1.25` → local server `192.168.1.50`, both inside `192.168.1.0/24`	Sensor `192.168.1.25` → **router / default gateway** → another network
Stays in the local network	**The router forwards IP traffic between different networks**

Private and public addresses

Private IP	**Public IP**
Used **within local networks**	Used for communication on the **public Internet**
Not directly routable on the public Internet	**Globally routable**
Example: 192.168.1.25	Assigned to an Internet-facing network/interface

The **router** connects the local (private) network to the Internet (public) and uses a **public IP** for Internet communication.

NAT — Network Address Translation

> **NAT** translates addresses between a **private network and an external network**, at the **network boundary**.

Why NAT is useful: many local devices can use **private IP addresses**, still communicate with Internet services, and it **helps conserve public IPv4 addresses**. With NAT, a home, office or university can use

private IPv4 addresses while still accessing the Internet.

IPv6

- Example: `2001:db8:abcd:0012::25` — **128 bits**, **8 groups of 16 bits**, hexadecimal, colon-separated (4 hex digits × 4 bits = 16 bits).
- Shortening rules: **leading zeros can be removed**, and **consecutive zero groups become** `::`. `2001:0db8:0000:0000:0000:ff00:0042:8329` → `2001:db8::ff00:42:8329`
- It provides a **far larger address space** and is designed to support continued Internet growth, since large-scale IoT involves very large numbers of devices. **It does not mean every IoT device must use IPv6 directly.**

	IPv4	**IPv6**
Address size	**32 bits**	**128 bits**
Format	Decimal, dotted	Hexadecimal, colon-separated
Example	192.168.1.25	2001:db8::25
Address space	Limited	Extremely large

> **IPv6 does not automatically solve every IoT networking problem.** Constrained devices still have **battery limitations, small memory and limited processing**.

The constrained-network problem → 6LoWPAN

Standard IPv6 communication creates **significant overhead** on constrained devices and low-power links — it simply **does not fit**, so it **requires adaptation**.

> **6LoWPAN** = **IPv6 over Low-power Wireless Personal Area Networks**: an **adaptation layer** that enables **IPv6 packets to be transmitted over low-power, constrained wireless networks**.

Key functions:

1. **Header compression** — omitting redundant header fields to reduce the standard **40-byte IPv6 header to as few as 2–7 bytes**.
2. **Fragmentation and reassembly** — splits large IPv6 packets into smaller fragments before transmission, and reassembles them at the receiving node.
3. **Mesh routing adaptation** — supports link- or network-layer routing across low-power wireless **mesh** topologies.

Architecture: low-power sensors (6LoWPAN nodes) communicate over the constrained wireless network; the **6LoWPAN gateway / border router** connects that mesh to the **global IPv6 Internet**. Once the packet enters the wide IP network, it continues as **ordinary IPv6**.

Ports and the transport layer

A single host can run **many services**, all sharing one IP address. The OS **finds the destination port number** in the incoming packet and **forwards data to the corresponding service**:

Destination	Service
`203.0.113.10:80`	Web browser / HTTP
`203.0.113.10:1883`	**MQTT** client
`203.0.113.10:5432`	Database

> **Transport layer** = **process-to-process (end-to-end)** communication between application processes on different hosts, ensuring data reaches the **specific software application**. **Network layer** = **host-to-host** routing.

Functions of the transport layer:

- **Application multiplexing through ports** — port numbers identify which application process the data belongs to.
- **Segmentation** — splits large application messages into smaller transport-layer segments.
- **Delivery behavior** — end-to-end delivery using **IP addresses + port numbers**.

TCP — Transmission Control Protocol

> TCP creates a **logical connection ("handshake")** before application data is exchanged: **[1] SYN** (connection request) → **[2] SYN-ACK** (received and accepted) → **[3] ACK** (connection confirmed).

Why the connection exists: communication **state** is established at both endpoints, and **sequence numbers** are coordinated for ordered delivery.

Four ideas in TCP:

1. **Sequence/order** — data is numbered as segments and placed in the correct order.
2. **Acknowledgment** — the receiver confirms received data; the ACK number is the **next expected byte** (cumulative ACK).
3. **Retransmission** — if the expected ACK doesn't arrive, the sender **retransmits**.
4. **Duplicate handling** — TCP can detect and **ignore duplicates**.

UDP — User Datagram Protocol

> **UDP** = a **simple, high-speed** protocol that sends data **without establishing a connection** or checking whether data arrives.

Characteristics:

- **Connectionless** — no handshake.
- **Unreliable delivery** — does not track lost, delayed or duplicated packets.
- **No ordering** — packets can take different routes.
- **No flow control** — sends as fast as the application generates data, even if the receiver is congested.

TCP vs UDP in one line: TCP = **reliable and ordered, with overhead**; UDP = **lightweight and fast, with no guarantees**.

Part 2 — Request-Response Communication

Why transport alone is insufficient

TCP or UDP moves data, **but how does an application express "give me the temperature" or "change this setting"?**

TCP can deliver `101010001101...`, but it does **not** know whether it means temperature, whether it is a request, whether it is a command, or which resource is being requested.

> **Application protocols** define the rules and interaction for application-to-application communication, so devices can **request resources** ("give me temperature") and **change settings** ("set the interval"). **The application protocol gives context to the exchange.**

Examples: `HTTP: GET /api/sensors/12/temperature` (retrieve a resource); `CoAP: GET /temperature` (tells the application it is a request).

Request-response and resource-oriented communication

- **Request-response model:** the **client initiates** the interaction.
- **Resource-oriented communication** structures application data and hardware functions into **distinct, uniquely addressable paths (URIs)**, so a client can request a **specific resource**.

HTTP in IoT

> **HTTP** = an **application-layer** protocol running over **TCP/IP**, structuring data exchange between client and server through a **request-response** model.

- **IoT / web client:** initiates by establishing a **TCP connection** and sending an **HTTP request**.
- **Web / API server:** processes the request, executes the action, and returns a **response containing a status code and payload**.

Anatomy of an HTTP request:

Part	Meaning
`GET`	**Action** — what to do with the resource (e.g. retrieve)
`/api/sensors/12/temperature`	**Resource** — the specific resource on the server
`Host`	**Destination service** — which server to send the request to

Common HTTP methods:

Method	Meaning in an IoT example
GET	Read sensor status
POST	Submit data / create an action
PUT	Update a resource/configuration
DELETE	Remove a resource/configuration

Example — reading sensor data: request path (client → server), then the server (1) reads sensor 12's temperature, (2) gets the latest value (**24.3 °C**), (3) sends a **JSON response** back.

Example — actuator command: the application sends a command to configure a device (a fan). The server **receives the command, authenticates and validates it, forwards it to the device, and updates the device configuration**. These are the **two information flow directions in IoT**: data up, commands down.

HTTP status codes:

Code	Status
200	OK
201	Created
400	Bad request
404	Not found
500	Server error

Example: `GET /sensors/99/temperature` when sensor 99 doesn't exist → **404 Not found**.

CoAP — Constrained Application Protocol

> **CoAP** = a **lightweight application protocol for constrained IoT environments**.

Characteristics: lightweight, **resource-oriented**, a **request-response model conceptually similar to HTTP**, and it **commonly uses UDP**.

	HTTP	**CoAP**
Interaction model	Request-response	Request-response
Typical transport	**TCP**	**UDP** commonly
Message representation	Text-oriented HTTP messages	**Compact binary**
Typical environment	Web / cloud / API systems	**Constrained IoT**
Resource model	Yes	Yes

CoAP example — a constrained room sensor. The outgoing path of a battery-powered room-temperature sensor, layer by layer:

Layer	What it does
Application data	Creates sensor data
CoAP	Defines the application message
UDP	Provides lightweight transport using ports
IPv6	Adds source and destination addressing
6LoWPAN	Adapts IPv6 for the constrained link
IEEE 802.15.4	Transmits data over the low-power wireless link

Incoming data is processed through the stack **in the reverse direction**.

Part 3 — MQTT and Publish-Subscribe

The problem with request-response: polling

> **Polling** = a system continuously asking a device or resource **if it has new data**, even when nothing is happening.

Analogy from the slides: **walking to the front door again and again to check whether the package has arrived** — exhausting.

With many sensors and applications, polling creates **repeated requests, unnecessary traffic, and poor scaling**.

Publish-subscribe model

1. **Publisher** sends a message **to a topic**.
2. **Broker** receives and **routes the message by topic**.
3. **Subscriber** receives messages for the **matching topic**.

Decoupling

Direct (request-response)	Broker-based (pub/sub)
Sensor must know **all destinations**	Sensor communicates **mainly with the broker**
Tighter coupling, more direct connections	**Decoupling** between sensors and applications
Harder to scale as applications increase	**Easier distribution**; adding new applications is simpler

MQTT architecture

> **MQTT** (Message Queuing Telemetry Transport) = a **lightweight publish-subscribe messaging protocol** used in IoT. **Sensors publish** data to a **broker**; **applications subscribe** to the data they are interested in. **Publishers and subscribers do not need to know each other directly.**

Connection and message flow (MQTT typically runs on top of **TCP/IP**):

1. Device establishes a **TCP connection** to the broker.
2. Client sends **CONNECT**.
3. Broker replies **CONNACK**.
4. Messaging begins — the client can **PUBLISH** messages and **SUBSCRIBE** to topics.

MQTT topics

Examples: `factory/machine01/temperature`, `factory/machine01/vibration`, `factory/machine01/status`, `factory/machine02/temperature` — a **hierarchy**: `factory` → `machine01` → `temperature`.

Poor topics	Better topics
`Sensor1`, `Sensor2`, `Data3`	`factory/line1/machine01/temperature`, `factory/line1/machine01/vibration`

Good topic design provides: an understandable hierarchy, scalable organization, easier subscription, and separation of measurement types.

Wildcards

Wildcards are used **in subscription topic filters**. Without them, a dashboard wanting every machine's temperature would need a **separate subscription per machine**.

Single-level `+` — matches **exactly one** topic level. `factory/+/temperature` breaks into `factory` / `+` / `temperature`:

- **Matches:** `factory/machine01/temperature`, `factory/machine02/temperature`, `factory/machine99/temperature`
- **Does not match:** `factory/machine01/vibration` (final level isn't temperature), `factory/line1/machine01/temperature` (too many levels)
- `factory/machine01/+` matches `…/temperature`, `…/vibration`, `…/status`

Multi-level `#` — matches **zero or more remaining** levels. `factory/#` matches `factory`, `factory/machine01`, `factory/machine01/temperature`, `factory/line1/machine02/status`, `factory/buildingA/room3/humidity` — **the whole subtree**.

Subscription	Meaning
`factory/+/temperature`	Temperature from **any one machine**
`factory/machine01/+`	**Any one-level item** from machine01
`factory/#`	**Everything** under factory

> **`+` replaces one level; `#` replaces all remaining levels.**

Worked example. Broker has `factory/machine01/temperature`, `factory/machine01/vibration`, `factory/machine02/temperature`, `factory/machine02/status`, `office/room101/temperature`:

Subscription	Receives
A: `factory/+/temperature`	`factory/machine01/temperature`, `factory/machine02/temperature`
B: `factory/machine01/+`	`factory/machine01/temperature`, `factory/machine01/vibration`
C: `factory/#`	all four `factory/...` topics (**not** `office/room101/temperature`)

Why wildcards matter: 500 machines × 3 measurements = **1,500 topics**. A dashboard needing only temperatures would need **500 separate subscriptions**; with `factory/+/temperature` it needs **one**. When `machine501` is added later, the dashboard **receives it automatically** — no new subscription. This gives MQTT easier configuration, better scalability, simpler dashboards, and less application-side subscription management.

Topic vs payload

Topic	Payload
`factory/machine01/temperature`	`{ "value": 68.4, "unit": "C" }`
Message classification / routing	**The actual information**

MQTT does not require JSON — the payload is **application-defined binary data**.

Complete flow: the sensor knows the **broker address, topic and payload**; it does **not** need to know the dashboard, database or alarm-system addresses. **The sensor sends data to the broker, and the broker distributes it to interested applications.**

MQTT QoS (Quality of Service)

> QoS defines **MQTT-level delivery semantics**, to ensure reliable delivery over unstable networks. IoT networks experience **drops, latency and interference**, and QoS lets you choose the balance between **reliability, speed and network cost**.

QoS	Delivery behavior	How it works	Best for	Example	Trade-off
0	**At most once**	PUBLISH sent once; **no acknowledgment**, no MQTT-level redelivery	Frequent, non-critical data	Temperature telemetry	**Lowest overhead, possible loss**
1	**At least once**	Publisher waits for **PUBACK**; retransmits if it doesn't arrive	Important messages	Fire alarm, machine fault alarm	**More reliable, duplicates possible**
2	**Exactly once**	**PUBLISH → PUBREC → PUBREL → PUBCOMP**	Messages where duplicate processing must be avoided	Critical control message, financial transaction	**Highest overhead**

QoS 0 is fine for periodic telemetry **because the next reading will arrive soon**.

Retained messages and Last Will

- **Retained message:** solves the **new-subscriber problem** by storing and delivering the **latest** message on a topic to new subscribers. It is the **latest retained state, not the complete history** — the broker keeps only the **most recent** retained message per topic.
- **Last Will:** a **predefined emergency message**, defined when the client connects, published **only if the client disconnects unexpectedly**. Example: topic `factory/machine01/status`, payload `offline` — if the device crashes or loses network, the broker publishes `factory/machine01/status = offline`.

Comparison: HTTP vs CoAP vs MQTT

Question	HTTP	CoAP	MQTT
Main model	Request-response	Request-response	**Publish-subscribe**
Common transport	TCP	**UDP**	TCP
Architecture	Client-server	Client-server / resource	**Broker-based**
Strength	Web/API integration	Constrained resources	Telemetry / event distribution
Typical IoT example	Cloud REST API	Constrained sensor	Continuous sensor messaging

> **They are not mutually exclusive — a real IoT system may use more than one.**

Trace a message from sensor to cloud

```
Physical temperature -> Sensor reading -> Structured data -> JSON payload   (week 3)
Wi-Fi connectivity -> Access point                                          (week 4)
IP -> TCP -> MQTT -> Topic factory/machine01/temperature -> Broker -> Dashboard   (week 5)
```

Layer	What it solves
Wi-Fi	Local connectivity
IP	Network addressing/delivery
TCP	Reliable transport stream
MQTT	Application messaging and publish-subscribe distribution
JSON	Data representation

Selecting a communication approach

1. **What interaction is needed?** Read/change a resource → **request-response**. Continuous telemetry/events → **publish-subscribe**.
2. **Is the endpoint constrained?** Very constrained device/network → **CoAP**. Web/cloud integration → **HTTP**. Brokered telemetry → **MQTT**.
3. **What delivery behavior is required?**

Requirement	Mechanism
Reliable, ordered transport	**TCP**
Lightweight transport without built-in reliability	**UDP**
MQTT publication delivery assurance	**MQTT QoS 0/1/2**

Summary stack

Layer	Protocols	Role
Application	HTTP / CoAP / MQTT	Where application-specific data is generated
Transport	TCP / UDP	End-to-end **process-to-process** communication using **port numbers**
Network	IPv4 / IPv6	**Host-to-host** addressing and internetwork routing
Adaptation (where needed)	6LoWPAN	Between the network layer and **constrained** physical links
Connectivity	Ethernet / Wi-Fi / 802.15.4 / cellular	Physical and data link layers

Checkpoints

1. "A factory temperature sensor publishes telemetry through Wi-Fi to an MQTT broker." Stack: **Wi-Fi → IP → TCP → MQTT.** Wi-Fi gives connectivity; **IP reaches the broker's host**; **TCP provides reliable transport** between applications; **MQTT provides publish-subscribe messaging**.

2. "Battery-powered room sensor. IEEE 802.15.4. Very small messages. We want to request its temperature resource." Stack: **IEEE 802.15.4 → 6LoWPAN → IPv6 → UDP → CoAP.** The constrained device and link have different resource and communication characteristics, so the lightweight, request-response path is used instead of TCP/HTTP.

Most important for exams

- **Layer questions:** connectivity = *how does the device access the network*; **IP = where should data go (which host)**; **transport = which application (port) and how delivered**; **application protocol = how applications exchange information**. Know the **package-delivery analogy**.
- **IPv4 = 32 bits, dotted decimal; IPv6 = 128 bits, hex, colon-separated** with `::` for consecutive zero groups. **IPv6 doesn't fix constrained-device limits.**
- **Private vs public IP, and NAT** at the network boundary (conserves public IPv4 addresses).
- **6LoWPAN** = IPv6 over low-power networks. Three functions: **header compression (40 bytes → 2–7)**, **fragmentation/reassembly**, **mesh routing adaptation**. The **border router** joins the mesh to the IPv6 Internet.
- **Transport = process-to-process; network = host-to-host.** Ports multiplex applications (80 HTTP, **1883 MQTT**).
- **TCP:** three-way handshake (**SYN, SYN-ACK, ACK**) and four ideas — **sequence/order, acknowledgment, retransmission, duplicate handling**. **UDP:** connectionless, unreliable, unordered, no flow control.
- **Why an application protocol is needed:** TCP delivers bits but doesn't know what they mean.
- **HTTP:** methods **GET/POST/PUT/DELETE** and their IoT meanings; status codes **200, 201, 400, 404, 500**; runs over **TCP**.
- **CoAP:** lightweight, resource-oriented, request-response like HTTP, **over UDP**, **compact binary**, for constrained IoT.
- **Polling problem → publish-subscribe**: publisher → **topic** → **broker** → subscriber, which **decouples** sensors from applications.
- **MQTT flow:** TCP connect → **CONNECT** → **CONNACK** → **PUBLISH / SUBSCRIBE**.
- **Wildcards: `+` = exactly one level, `#` = all remaining levels.** Be ready to say which topics a filter matches.
- **Topic ≠ payload** (routing vs information); **MQTT does not require JSON**.
- **QoS 0 / 1 / 2 = at most once / at least once / exactly once**, with PUBACK for 1 and **PUBREC-PUBREL-PUBCOMP** for 2, plus the trade-off of each.
- **Retained message** (latest state for new subscribers) vs **Last Will** (published when a client disconnects unexpectedly).
- **HTTP vs CoAP vs MQTT comparison table**, and the two checkpoint stacks (**Wi-Fi/IP/TCP/MQTT** and **802.15.4/6LoWPAN/IPv6/UDP/CoAP**).